06 · Worklist and reviewing flags

Open flags grouped by person, worst first; review in one click, or dismiss with a reason.

New to the syntax? Read **00 · How to read this code** first; this guide explains what is specific to this feature.

What this feature does

The **Alerts** page for workers lists people, not readings: one card per citizen with all their open flags, worst first. A worker can mark the person reviewed, dismiss with a reason and optional note, book a visit, or open the record. A second tab shows every flag raised in the last 14 days. Search accepts a name or an HN reference such as HN-1A2B3C4D.

Clearing a flag records who did it, when, the decision, the reason and the note, so every cleared flag can be audited.

Files involved

Only the files this feature uses.

```
backend/app/
├── api/routes/phc.py       GET /phc/worklist, POST /phc/worklist/citizens/{id}/resolve,
│                           POST /phc/queue/{id}/seen, GET /phc/alerts (recent flags)
└── schemas/phc.py          Resolve (the review body), WorkItem, FlagBrief
frontend/src/
├── pages/phc/PhcAlerts.tsx       the page, search, locality filter, tabs
├── components/app/phc.tsx        CitizenWorkCard, ResolveControls, BookVisitForm
└── services/phc.ts               getWorklist(), resolveCitizen(), markSeen(), getFeed()
```

How the pieces connect

1. The page calls `GET /phc/worklist` with the search text and locality; the backend groups open flags by citizen.
2. The worker clicks **Mark seen** or **Dismiss...** on a card; `ResolveControls` sends the decision.
3. `resolve_citizen` checks the citizen is in the catchment and clears all their open flags in one UPDATE.
4. The page reloads; the card disappears and the menu badge drops on its next check.

Code walkthrough

Every excerpt below is copied straight from the file, with its real line numbers, when this guide was generated.

1. `backend/app/schemas/phc.py` · Resolve

The request body for a review. The rules are enforced here, before any route code runs.

```
87  class Resolve(BaseModel):
88      resolution: FlagResolution = FlagResolution.reviewed
89      dismiss_reason: FlagDismissReason | None = None
90      note: Annotated[str, StringConstraints(strip_whitespace=True, max_length=500)] | None = None
91
92      @model_validator(mode="after")
93      def _reason_matches(self) -> "Resolve":
94          if self.resolution == FlagResolution.dismissed and self.dismiss_reason is None:
95              raise ValueError("Say why the flag is being dismissed")
96          if self.resolution == FlagResolution.reviewed and self.dismiss_reason is not None:
97              raise ValueError("A dismiss reason only applies when dismissing")
98          return self
```

What the code means

- Fields with defaults are optional: an empty body means "reviewed".
- `Annotated[str, StringConstraints(strip_whitespace=True, max_length=500)] | None`: an optional note, trimmed, at most 500 characters.

- `@model_validator(mode="after")` runs after the fields are read and can check them together: a dismissal **must** have a reason, a review must not.
- `raise ValueError(...)` becomes a 422 error with that message.

2. `backend/app/api/routes/phc.py` · _apply

```
330 def _apply(processed_query, worker: User, data: Resolve):
331     return processed_query.values(
332         reviewed_at=utcnow(),
333         reviewed_by=worker.id,
334         resolution=data.resolution,
335         dismiss_reason=data.dismiss_reason,
336         resolution_note=data.note,
337     )
```

What the code means

- Takes an UPDATE query and adds the review values. Shared by "clear one flag" and "clear all of a person's flags", so both save the same fields.

3. `backend/app/api/routes/phc.py` · resolve_citizen

```
460 @router.post("/worklist/citizens/{user_id}/resolve", response_model=Cleared)
461 async def resolve_citizen(
462     user_id: uuid.UUID,
463     data: Resolve | None = None,
464     worker: User = Depends(require_catchment_worker),
465     db: AsyncSession = Depends(get_session),
466 ) -> Cleared:
467     citizen = await get_catchment_citizen(db, worker, user_id)
468     result = await db.execute(
469         _apply(update(ProcessedData).where(ProcessedData.user_id == citizen.id, *_open_flag()), worker, data
                or Resolve())
470     )
471     await db.commit()
472     return Cleared(cleared=result.rowcount or 0)
```

What the code means

- `data: Resolve | None = None`: the body is optional; `data or Resolve()` uses the default "reviewed".
- `get_catchment_citizen` gives 404 for anyone outside the catchment.
- One UPDATE clears every open flag of that person; `result.rowcount` reports how many.

4. `backend/app/api/routes/phc.py` · mark_seen (clear one flag)

```
341 async def mark_seen(
342     processed_id: uuid.UUID,
343     data: Resolve | None = None,
344     worker: User = Depends(require_catchment_worker),
345     db: AsyncSession = Depends(get_session),
346 ) -> Cleared:
347     data = data or Resolve()
348     processed = await db.scalar(
349         select(ProcessedData)
350         .where(
351             ProcessedData.id == processed_id,
352             ProcessedData.is_anomaly.is_(True),
353             ProcessedData.user_id.in_(catchment_ids(worker)),
354         )
355         .with_for_update()
356     )
357     if processed is None:
358         raise HTTPException(status.HTTP_404_NOT_FOUND, CASE_NOT_FOUND)
359     if processed.resolution is not None:
360         raise HTTPException(status.HTTP_409_CONFLICT, "This case has already been cleared.")
361     await db.execute(_apply(update(ProcessedData).where(ProcessedData.id == processed.id), worker, data))
362     await db.commit()
363     return Cleared(cleared=1)
364
```

What the code means

- `.with_for_update()` locks the row until the transaction ends, so two workers clicking at the same moment cannot both clear it.
- Already cleared gives 409 Conflict instead of silently overwriting the first decision.

5. `backend/app/api/routes/phc.py` · worklist (grouping)

```
370  async def worklist(
371      locality: str | None = Query(None, max_length=120),
372      q: str | None = Query(None, max_length=80),
373      limit: int = Query(50, ge=1, le=100),
374      worker: User = Depends(require_catchment_worker),
375      db: AsyncSession = Depends(get_session),
376  ) -> WorklistOut:
377      now = now_utc()
378      grouped = (
379          select(
380              ProcessedData.user_id,
381              func.count().label("n"),
382              func.max(ProcessedData.anomaly_score).label("worst"),
383              func.min(ProcessedData.processed_at).label("first"),
384              func.max(ProcessedData.processed_at).label("last"),
385          )
386          .join(User, User.id == ProcessedData.user_id)
387          .where(ProcessedData.user_id.in_(catchment_ids(worker)), *_open_flag())
388          .group_by(ProcessedData.user_id)
389      )
390      if locality:
391          grouped = grouped.where(_locality_filter(locality))
392      if q and q.strip():
393          grouped = grouped.where(_search(q))
394      total = await _count(db, grouped)
395      groups = (
396          await db.execute(grouped.order_by(func.max(ProcessedData.anomaly_score).desc(), func.max(ProcessedDa
                ta.processed_at).desc()).limit(limit))
397      ).all()
398      ids = [g.user_id for g in groups]
399      if not ids:
```

What the code means

- `.group_by(ProcessedData.user_id)` makes one row per citizen with: number of open flags (`func.count()`), worst score (`func.max`), first and last flag times.
- `.label("n")` names a computed column so it can be read as `g.n`.
- Sorted by worst score, then most recent: the most urgent person comes first.
- Later lines (not shown) load up to 10 flags per person and any visit already booked by this worker.

6. `frontend/src/components/app/phc.tsx` · ResolveControls

```
73  export function ResolveControls({
74    onResolve,
75    reviewLabel = "Mark seen",
76  }: {
77    onResolve: (body: ResolveBody) => Promise<unknown>;
78    reviewLabel?: string;
79  }) {
80    const [mode, setMode] = useState<"idle" | "dismiss">("idle");
81    const [reason, setReason] = useState<DismissReason>("contacted");
82    const [note, setNote] = useState("");
83    const [busy, setBusy] = useState(false);
84    const [error, setError] = useState<string | null>(null);
85
86    const run = async (body: ResolveBody) => {
87      setBusy(true);
88      setError(null);
89      try {
90        await onResolve(body);
91      } catch (e) {
92        setError(errorMessage(e));
93        setBusy(false);
94      }
95    };
```

```
96
    ... (33 more lines in the file: the buttons follow)
```

What the code means

- `onResolve` is a **callback prop**: the parent decides what resolving means (one flag or one person), this component only collects the decision.
- `mode` switches between the two buttons and the dismiss form (`"idle" | "dismiss"` is a union type: only those two values are allowed).
- `run` sets `busy`, awaits the request, and on failure shows the error and re-enables the buttons.

7. `frontend/src/pages/phc/PhcAlerts.tsx` · useDebounced

Search should not send a request for every key press.

```
10  function useDebounced(value: string, ms = 300) {
11    const [out, setOut] = useState(value);
12    useEffect(() => {
13      const t = window.setTimeout(() => setOut(value), ms);
14      return () => window.clearTimeout(t);
15    }, [value, ms]);
16    return out;
17  }
```

What the code means

- Each change starts a 300 ms timer; the clean-up function cancels the previous timer, so only a pause in typing updates the value.
- `window.setTimeout` / `clearTimeout` schedule and cancel the update.

Syntax used in this feature

Code	What it means
`@model_validator(mode="after")`	Pydantic check that looks at several fields together.
`.with_for_update()`	Lock the selected rows until commit (SQL SELECT ... FOR UPDATE).
`.group_by(col)`	One result row per value of col.
`func.max(col).label("worst")`	A computed column with a name.
`"idle" | "dismiss"`	A TypeScript type that allows only those two strings.